ImpactLedger: Corrected Implementation Report
(Code-Grounded Revision of draft7)

Project Documentation Update

April 22, 2026

Abstract

This report replaces mismatched claims from the prior draft with statements grounded in the current repository implementation. The corrected system is a Next.js App Router application backed by Supabase/PostgreSQL, with role-aware modules for donor, volunteer, and organization-admin workflows. Donation flows are integrated with Razorpay checkout and webhook reconciliation, with invoice dispatch and receipt generation handled by server modules.

1 Scope and Correction Strategy

The objective is to align documentation with implemented behavior rather than preserve legacy assumptions. We used:

- code inventory of pages, APIs, server modules, and schema;
- extracted claim clusters from `draft7.pdf`;
- mismatch normalization into a claim-correction matrix.

The matrix source is provided in `report/claim_mismatch_matrix.csv`.

2 Implemented Tech Stack

Frontend and runtime:

- Next.js + React (App Router).
- Route handlers under `src/app/api/**/route.ts`.

Data and auth:

- Supabase/PostgreSQL schema in `supabase/schema.sql`.
- Supabase auth with tenant role resolution in `src/lib/server/auth.ts`.

Payments and receipts:

- Razorpay order creation and signed webhook verification.
- Invoice dispatch via SMTP and generated receipt PDFs.

Tooling scripts:

- `npm run lint`, `npm run test`, `npm run build`.

3 Functional Scope (Implemented Features)

3.1 Public

Campaign listing and details, donation entry points, transparency view, impact page, and policy/legal pages.

3.2 Donor

Dashboard and donation history endpoints/pages.

3.3 Volunteer

Dashboard, assignments, logs, resources, and verification pages with corresponding APIs.

3.4 Organization Admin

Dashboard, campaigns, donors, operations, reports, analytics, and settings APIs/pages.

4 Route and API Surface

The canonical route inventory is maintained in `report/canonical_inventory.md`. In summary:

- App pages are defined under `src/app/**/page.tsx`.
- APIs are defined under `src/app/api/**/route.ts`.
- Payment APIs include checkout, verification, Razorpay webhook; Stripe webhook endpoint remains but is explicitly deprecated with HTTP 410.

5 Data Model

The application uses a multi-tenant relational model in Supabase/PostgreSQL. Core domains include:

- tenant and membership tables for RBAC;
- campaign, donor, donation, and financial ledger tables;
- volunteer profile/assignment/report tables;
- audit and webhook event logging tables;
- reporting views for campaign funding and transparency.

6 Architecture and Design

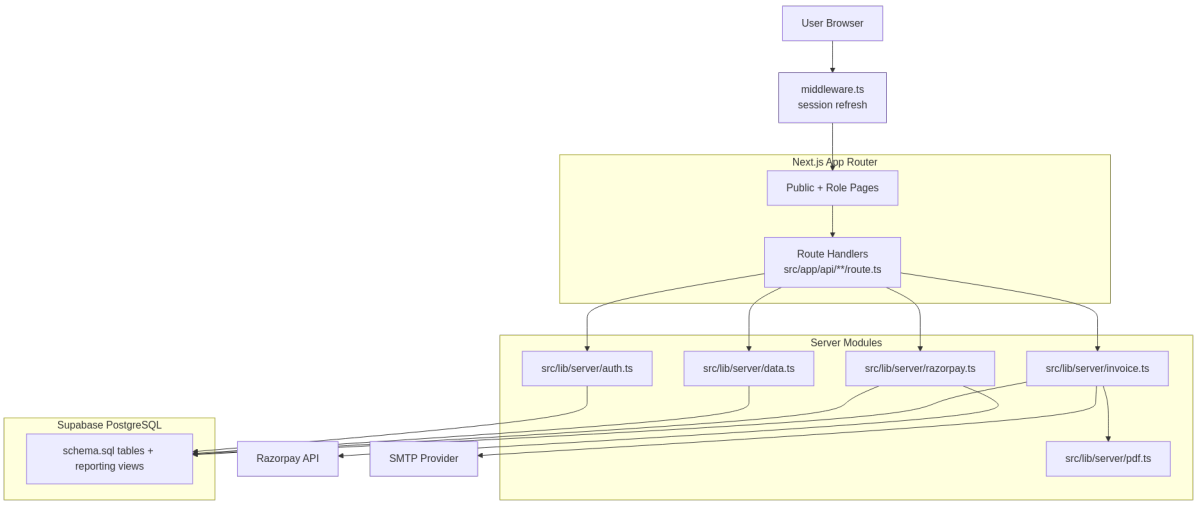

Figure 1: System architecture derived from current code modules and integrations.

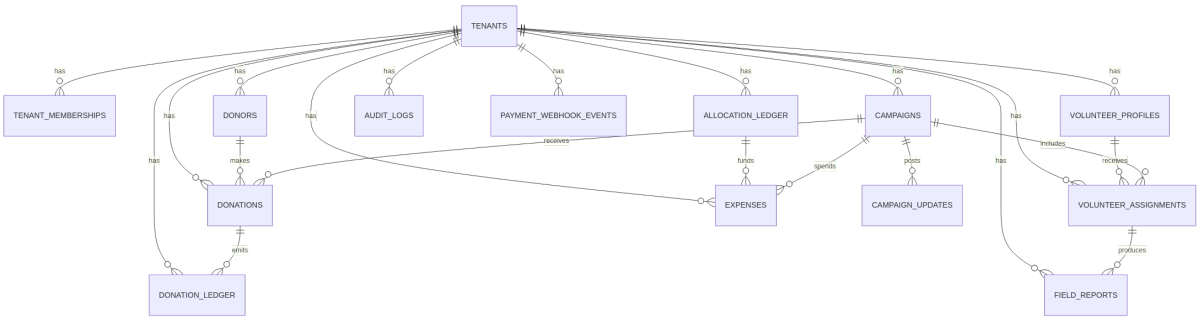

Figure 2: Entity-relationship overview from `supabase/schema.sql`.

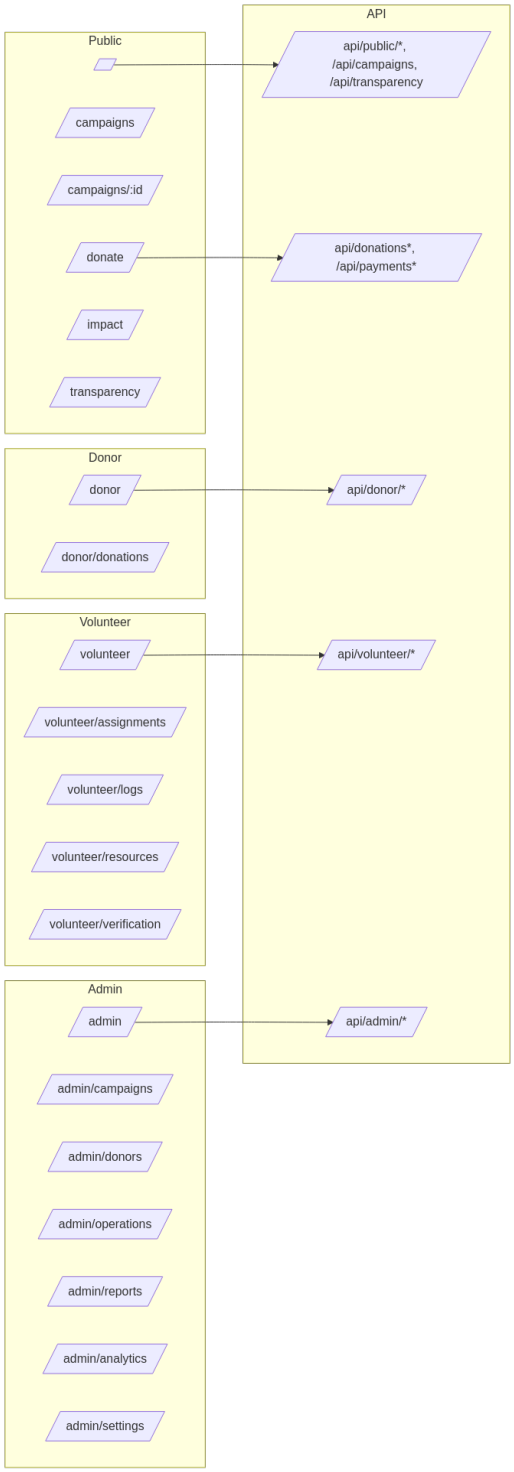

Figure 3: Route and API map for public and role-scoped modules.

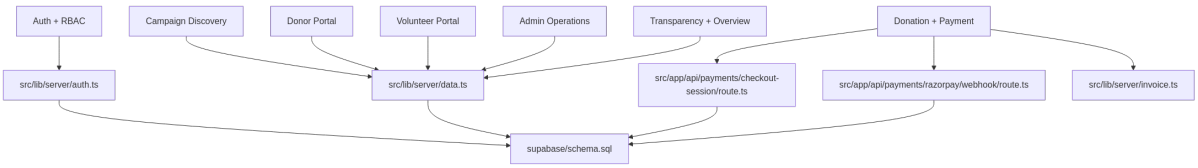

Figure 4: Feature-to-module map showing service and schema dependencies.

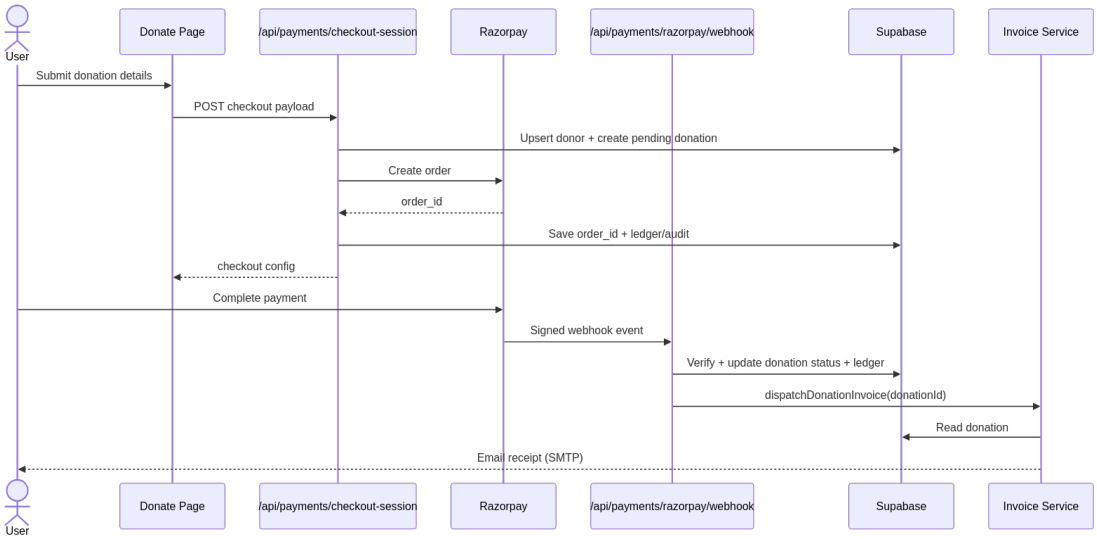

Figure 5: Donation flow sequence: checkout, payment confirmation, and receipt dispatch.

7 Methodology (Evidence-First Documentation)

This corrected report does not assert historical sprint metrics or stakeholder-process details unless directly evidenced in repository artifacts. The methodology here is implementation-first:

1. identify claims in the previous document;
2. map each claim to code evidence or mark as unsupported;
3. rewrite sections from implemented modules and schemas;
4. regenerate architecture and data diagrams from code-grounded structures.

8 Requirements (Code-Backed)

8.1 Functional requirements

Derived from concrete routes and handlers:

- role-based access to donor, volunteer, and admin modules;
- campaign discovery and detail retrieval;

- donation creation and status tracking;
- payment reconciliation via signed webhook processing;
- ledger and audit event capture;
- volunteer assignment/report workflows;
- admin operations, reporting, and analytics endpoints.

8.2 Operational requirements

- Supabase and application environment variables for auth and tenancy;
- Razorpay API keys and webhook secret for payment workflows;
- SMTP configuration for invoice email dispatch.

9 Implementation Notes

- Middleware: `middleware.ts` delegates session update handling.
- Auth context and role checks: `src/lib/server/auth.ts`.
- Data aggregation and role-specific data loaders: `src/lib/server/data.ts`.
- Payment path:
  - checkout order creation: `/api/payments/checkout-session`;
  - webhook reconciliation: `/api/payments/razorpay/webhook`;
  - signature verification helper: `src/lib/server/razorpay.ts`.
- Invoice path:
  - dispatch: `src/lib/server/invoice.ts`;
  - PDF creation: `src/lib/server/pdf.ts`.

10 Results and Impact (Conservative Restatement)

The corrected document limits results statements to repository-verifiable signals:

- implemented route coverage across public and role-scoped modules;
- structured relational schema with audit and ledger tables;
- integrated payment and receipt pipeline with webhook persistence;
- available integration test suite for DB-backed API paths.

Where prior draft claims provided fixed performance/uptime/social-impact values without reproducible in-repo evidence, those claims are intentionally omitted or reframed.

11 Reproducibility

Diagram sources are maintained as Mermaid files in `report/diagrams/*.mmd`. Render with:

```
./report/render_diagrams.sh
```

Compile this report from repository root:

```
pdflatex -output-directory report report/impactledger_corrected.tex
```

Appendix: Source Index

- `report/claim_mismatch_matrix.csv`
- `report/canonical_inventory.md`
- `report/diagrams/*.mmd`
- `supabase/schema.sql`
- `supabase/migrations/20260415_add_reporting_views.sql`
- `src/lib/server/auth.ts`
- `src/lib/server/data.ts`
- `src/lib/server/razorpay.ts`
- `src/lib/server/invoice.ts`
- `src/lib/server/pdf.ts`
- `src/app/api/**/route.ts`